

Dart

Dart is an open-source general-purpose programming language developed by Google. It supports application development on both the client and server side. Dart is a dynamic, object-oriented language similar to Java, C, and JavaScript, featuring closures and block scope. The Dart language was released in 2011 but came into popularity after 2015 with Dart 2.0.

Features of Dart

Object Oriented Programming: Nowadays OOPS is considered as one of the most important features for many Programming languages including Java and C++. Now, Dart is following the same path of OOPS .

Fast & Smooth: Dart compiles to native code for speedy performance, ideal for mobile apps.

Easy to Learn: Similar to familiar languages like Java or Javascript, making it approachable for new developers.

One Code, Many Places: Develop for mobile, web, and even desktop with a single codebase (primarily with Flutter).

Flutter Power: Dart is the heart of Flutter, a popular framework for building beautiful and functional mobile apps.

Installation of Dart

Please install Flutter SDK, which includes the Dart SDK, so there's no need to install Dart separately. You have to Create a file named "your_filename.dart" in your IDE and start writing your program.

First Program in Dart

In dart, main() function is a predefined method and acts as the entry point to the application. A dart script needs the main() method for execution of the code. The program code goes like this:

```
main() {  
  print("Hello World! Hello Dart");  
}
```

output : Hello World! Hello Dart.

main(): it is the symbol of the main function that means the data entered in it is directly executed by the compiler.

print("Hello World!") : the role of print function is quite simple it just prints the data during the compilation of a program.

The dart program can also be compile through terminal by executing the code dart file_name.dart.

In every programming language comments play an important role for a better understanding of the code in the future or by any other programmer. Comments are a set of statements that are not meant to be executed by the compiler. They provide proper documentation of the code.

There are three type of comments;

Dart Single line Comment.

Dart Multiline Comment.

Dart Documentation Comment.

Dart Single line Comment: Dart single line comment is used to comment a line until line break occurs. It is done using a double forward-slash (//). (Eg: // This is a single line comment).

Dart Multi-Line Comment: Dart Multiline comment is used to comment out a whole section of code. It uses `/*` and `*/` to start and end a multi-line comment respectively. (Eg.

```
/*  
These are  
multiple line  
of comments  
*/ ).
```

Dart Documentation Comment: Dart documentation comments are used to reference packages or projects, can be written using `///
*/` (C# style) or `/** ... */` (JavaDoc style). It's

preferred to use "///" for better readability, especially when listing items, and they are recommended for documenting public APIs.

(Eg.

```
/// This is  
/// named as a  
/// document comment ).
```

Variables

Variables are the places used to store value in the program. There are different types of variables where you can keep different kinds of values. Here is an example of creating a variable and initializing it.

Syntax : type variable_name; (Eg. var name;).

In dart, there are three types of Variables.

Static Variable

Dynamic Variable

Final or const

- **Variable / Static Variable:**

Variables are also called data types. We will learn more about data types later.

String: For storing text value. (E.g. "Soft" [Must be in quotes]).

int: For storing integer value. (E.g. 10, -10, 8555 [Decimal is not included]).

double: For storing floating point values. (E.g. 10.0, -10.2, 85.698 [Decimal is included]).

num: For storing any type of number. (E.g. 10, 20.2, -20 [both int and double]).

bool: For storing true or false. (E.g. true, false [Only stores true or false values]).

var: For storing any value. (E.g. 'Soft', 12, 'z', true).

Syntax:

```
// declaring variables  
String name = "Soft";  
int number = 20;  
float height = 6.7;  
num age = 25; // used to store any numbers such as int,float  
num height = 5.9;  
bool isMarried = false;  
var name = "soft";  
var age = 20;  
var height = "5.9";
```

```
var isMarried = false;

// printing variables value
print("Name is $name");
print("Address is $address");
print("Age is $age");
print("Height is $height");
print("Married Status is $isMarried");
```

- **Dynamic Variable**

A dynamic variable can store any value at runtime, similar to var, but unlike var, it retains its dynamic type without being converted to a specific type upon assignment.

Syntax: dynamic variable_name; //can store any value to this dynamic variable.

- **Final And Const**

These keywords define constant variables in Dart, meaning their values cannot change throughout the code. They can be used with or without a data type.

Final: A variable can only be assigned once, but its value can be determined at runtime.

Const: A variable must be assigned a compile-time constant value and cannot be changed.

There are some Rules to create variable names,

1. Variable names or identifiers can't be the keyword.
2. Variable names or identifiers can contain alphabets and numbers.
3. A variable name cannot start with a number.
4. Blank spaces are not allowed in a variable name.
5. Special characters are not allowed except for the underscore (_) and the dollar (\$) sign.

Data Types

Data types shows the different kinds of data in your code, like numbers, text, and symbols. They tell you what type of value a variable can hold. Dart has several built-in data types.

Dart supports the following built-in data types :

Numbers - It represents numeric values

Strings - It represents a sequence of characters

Booleans - It represents Boolean values true and false

Lists - It is an ordered group of items

Maps - It represents a set of values as key-value pairs

Sets - It is an unordered list of unique values of same types

Runes - It represents Unicode values of String

Null - It represents null value

- **Numbers**

When you need to store numeric value on dart, you can use either int or double. Both int and double are subtypes of num. You can use num to store both int or double value.

Syntax:

```
// Declaring Variables
```

```
int num1 = 100; // without decimal point.
```

```
double num2 = 130.2; // with decimal point.
```

```
num num3 = 50;
```

```
num num4 = 50.4;
```

```
// For Sum
```

```
num sum = num1 + num2 + num3 + num4;
```

```
// Printing Info
```

```
print("Num 1 is $num1");
```

```
print("Num 2 is $num2");
```

```
print("Num 3 is $num3");
```

```
print("Num 4 is $num4");
```

```
print("Sum is $sum");
```

- **Strings**

String helps you to store text data. You can store values like "I love dart, New York 2140" in String. You can use single or double quotes to store string in dart.

Syntax:

```
// Declaring Values
```

```
String schoolName = "Diamond School";
```

```
String address = "New York 2140";
```

```
// Printing Values
```

```
print("School name is $schoolName and address is $address");
```

1. Multi-Line String In Dart

If you want to create a multi-line String in dart, then you can use triple quotes with either single or double quotation marks.

Syntax:

```
// Multi Line Using Single Quotes
String multiLineText = '''
This is Multi Line Text
with 3 single quote
I am also writing here.
''';
// Multi Line Using Double Quotes
String otherMultiLineText = """
This is Multi Line Text
with 3 double quote
I am also writing here.
""";
// Printing Information
print("Multiline text is $multiLineText");
print("Other multiline text is $otherMultiLineText");
```

In Dart, you can easily print strings and numbers together using the \$ sign for simple variables and \${} for expressions.

Eg:

```
single number : print("My age is $10");
multiple number : print("My height is ${5.6}").
```

Type Conversion In Dart

1. Convert String To Int In Dart:

You can convert String to int using `int.parse()` method. The method takes String as an argument and converts it into an integer.(Eg. `int intValue = int.parse(strvalue);`).

Syntax:

```
String strvalue = "1";
print("Type of strvalue is ${strvalue.runtimeType}");
int intValue = int.parse(strvalue);
print("Value of intValue is $intValue");
// this will print data type
print("Type of intValue is ${intValue.runtimeType}");
```

2. Convert Double To Int In Dart

You can convert double to int using the `toInt()` method.

Syntax:

```
double num1 = 10.01;
int num2 = num1.toInt(); // converting double to int

print("The value of num1 is $num1. Its type is ${num1.runtimeType}");
print("The value of num2 is $num2. Its type is ${num2.runtimeType}");
```

3. Convert String To Double In Dart

You can convert String to double using `double.parse()` method. The method takes String as an argument and converts it into a double.

Syntax :

```
String strvalue = "1.1";
print("Type of strvalue is ${strvalue.runtimeType}");
double doublevalue = double.parse(strvalue);
print("Value of doublevalue is $doublevalue");
// this will print data type
print("Type of doublevalue is ${doublevalue.runtimeType}");
```

4. Convert Int To String In Dart

You can convert int to String using the `toString()` method. Here is example:

Syntax :

```
int one = 1;
print("Type of one is ${one.runtimeType}");
String oneInString = one.toString();
print("Value of oneInString is $oneInString");
// this will print data type
print("Type of oneInString is ${oneInString.runtimeType}");
```

5. Convert Double To Int In Dart

You can convert double to int using the `toInt()` method.

Syntax:

```
double num1 = 10.01;
int num2 = num1.toInt(); // converting double to int

print("The value of num1 is $num1. Its type is ${num1.runtimeType}");
print("The value of num2 is $num2. Its type is ${num2.runtimeType}");
```

Booleans

In Dart, boolean holds either true or false value. You can write the bool keyword to define the boolean data type. You can use boolean if the answer is true or false.

Syntax:

```
bool isMarried = false;
print("Married Status: $isMarried");
output : false;
```

- **Lists**

The list holds multiple values in a single variable. It is also called arrays. If you want to store multiple values without creating multiple variables, you can use a list.

Syntax:

```
List<String> names = ["Raj", "John", "Max"];
print("Value of names is $names");
print("Value of names[0] is ${names[0]}"); // index 0
print("Value of names[1] is ${names[1]}"); // index 1
print("Value of names[2] is ${names[2]}"); // index 2
// Finding Length of List
int length = names.length;
print("The Length of names is $length");
```

- **Sets**

An unordered collection of unique items is called set in dart. You can store unique data in sets.

Syntax:

```
Set<String> weekday = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
print(weekday);
```


In a set, if you try to add a value that already exists, it won't be added again; instead, it simply overrides the previous instance, resulting in no change.

- **Maps**

In Dart, a map is an object where you can store data in key-value pairs. Each key occurs only once, but you can use same value multiple times.

Syntax:

```
Map<String, String> myDetails = {  
  'name': 'John Doe',  
  'address': 'USA',  
  'fathername': 'Soe Doe'  
};  
// displaying the output  
print(myDetails['name']);
```

- **Runes**

With runes, you can find Unicode values of String. The Unicode value of a is 97, so runes give 97 as output.

Syntax:

```
String value = "a";  
print(value.runes);  
output : 97
```

Operators In Dart

Operators in Dart perform mathematical and logical operations on variables, using symbols to indicate the type of operation. Before learning about them, it's helpful to understand a few key concepts.

Operands : It represents the data.

Operator : It represents how the operands will be processed to produce a value.

Types Of Operators

There are different types of operators in dart. They are as follows:

Arithmetic Operators

Increment and Decrement Operators

Assignment Operators

Logical Operators

Type Test Operators

1. Arithmetic Operators

Arithmetic operators are the most common types of operators. They perform operations like,

Addition : For adding two operands(+).

Subtraction : For subtracting two operands(-).

Multiplication : For multiplying two operands(*).

Division : For dividing two operands and give output in double(/).

Modulus : Remainder After Integer Division(%).

Syntax:

```
int num1=10;
```

```
int num2=3;
```

```
// arithmetic calculation
```

```
int sum=num1+num2;    // addition
```

```
int diff=num1-num2;    // subtraction
```

```
int unaryMinus = -num1; // unary minus
```

```
int mul=num1*num2;    // multiplication
```

```
double div=num1/num2; // division
```

```
int div2 =num1~/num2; // integer division
```

```
int mod=num1%num2;    // show remainder
```

```
print("The addition is $sum.");
```

```
print("The subtraction is $diff.");
```

```
print("The unary minus is $unaryMinus.");
```

```
print("The multiplication is $mul.");
```

```
print("The division is $div.");
```

```
print("The integer division is $div2.");
```

```
print("The modulus is $mod.");
```

2. Increment and Decrement Operators

With increment and decrement operators, you can increase and decrease values. If ++ is used at the beginning, then it is a prefix. If it is used at last, then it is postfix.

Pre Increment - Increase Value By 1. var = var + 1 Expression value is var+1(++a).

Pre Decrement - Decrease Value By 1. var = var - 1 Expression value is var-1(--a).

Post Increment - Increase Value By 1. $\text{var} = \text{var} + 1$ Expression value is $\text{var}(a++)$.
Post Decrement - Decrease Value By 1. $\text{var} = \text{var} - 1$ Expression value is $\text{var}(a--)$.

Syntax:

```
int a=0;  
int b=2;
```

```
a = ++a; // pre increment  
b = --b; // pre decrement  
print("The value of a is $a and b is $b"); // output : The value of a is 1 and b is 0.
```

```
// reset value to 0  
a = 0;  
b = 1;
```

```
a = a++; // post increment  
b = a--; // post decrement  
print("The value of a is $a and b is $b"); // output : The value of a is 1 and b is 0.
```

3. Assignment Operators

It is used to assign some values to variables. Here, we are assigning 24 to the age variable. Eg. $\text{int age} = 24$;

= : Assign a value to a variable.
+= : Adds a value to a variable.
-= : Reduces a value to a variable.
*= : Multiply value to a variable.
/= : Divided value by a variable.

Syntax:

```
double age = 24;  
age+= 1; // Here age+=1 means age = age + 1.  
print("After Addition Age is $age");  
age-= 1; //Here age-=1 means age = age - 1.  
print("After Subtraction Age is $age");  
age*= 2; //Here age*=2 means age = age * 2.  
print("After Multiplication Age is $age");  
age/= 2; //Here age/=2 means age = age / 2.  
print("After Division Age is $age");
```

4. Relational Operators

Relational operators are also called Comparison Operators. They are used to make a comparison.

Greater than(>) : Used to check which operand is bigger and gives result as boolean.

Less Than(<) : Used to check which operand is smaller and gives result as boolean.

Greater than or equal to(>=) : Used to check which operand is bigger or equal and gives result as boolean.

Less than or equal to(<=) : Used to check which operand is smaller or equal and gives result as boolean.

Equal to(==) : Used to check operands are equal to each other and gives result as boolean.

Not equal to(!=) : Used to check operand are not equal to each other and gives result as boolean.

Syntax:

```
int num1=10;
int num2=5;
//printing info
print(num1==num2);
print(num1<num2);
print(num1>num2);
print(num1<=num2);
print(num1>=num2);
```

5. Logical Operators

Logical Operators are used to compare values.

&& : This is 'and', return true if all conditions are true.

|| : This is 'or'. Return true if one of the conditions is true.

! : This is 'not'. return false if the result is true and vice versa.

Syntax:

```
int userid = 123;
int userpin = 456;
// Printing Info
print((userid == 123) && (userpin== 456)); // print true
print((userid == 1213) && (userpin== 456)); // print false.
print((userid == 123) || (userpin== 456)); // print true.
print((userid == 1213) || (userpin== 456)); // print true
print((userid == 123) != (userpin== 456));//print false
```

6. Type Test Operators

Type test operators are useful for checking types at runtime.

`is` : Gives boolean value true if the object has a specific type.

`!is` : Gives boolean value false if the object has a specific type.

Syntax:

```
String value1 = "Dart Tutorial";  
int age = 10;
```

```
print(value1 is String);  
print(age is !int);
```

User Inputs in Dart

In every programming language, you can provide input to the computer, but the methods for doing so can vary significantly

In Dart, for instance, you need to import the package `import 'dart:io';` to facilitate user input through the console. This allows you to read data from the standard input, enabling interactive programs that respond to user commands or data entries.

1. String User Input

Here, you can take string input from the user. By getting a String from a user we have to use `"stdin.readLineSync()"` for getting the input.

Eg:

```
print("Enter name:"); // output= Enter name  
String? name = stdin.readLineSync(); // Sneha  
print("The entered name is ${name}"); // The entered name is Sneha
```

2. Integer User Input

Here, You can take integer input to get a numeric value from the user without the decimal point. Here we need to parse the integer value which is `"int.parse(stdin.readLineSync())"`.

Eg:

```
print("Enter number:"); // Enter number:
int? number = int.parse(stdin.readLineSync()!); // 50
print("The entered number is ${number}"); // The entered number is 50
```

3. Floating Point User Input

Here, you can take floating point value input from the user. By getting of double from a user we have to use "double.parse(stdin.readLineSync()!)" for getting the input.

Eg:

```
print("Enter a floating number:"); // Enter a floating number:
double number = double.parse(stdin.readLineSync()!); // 55.5
print("The entered num is $number"); // The entered num is 55.5
```

String In Dart

A string is used to store text-based data, such as your name, address, or a book. It represents a sequence of characters, including letters, numbers, and special symbols. You can define a string using single, double, or triple quotes. While we've discussed strings before, they also offer many additional functionalities and properties worth exploring. These include methods for manipulating text, checking lengths, formatting, and searching within the string.

Eg:

```
String text1 = 'This is an example of a single-line string.';
String text2 = "This is an example of a single line string using double quotes.";
String text3 = """This is a multiline line
string using the triple-quotes.
This is tutorial on dart strings.
""";
```

Properties Of String

codeUnits: Returns an unmodifiable list of the UTF-16 code units of this string.

isEmpty: Returns true if this string is empty.

isNotEmpty: Returns false if this string is empty.

length: Returns the length of the string including space, tab, and newline characters.

Syntax:

```
String str = "Hi";
print(str.codeUnits); //Example of code units
print(str.isEmpty); //Example of isEmpty
print(str.isNotEmpty); //Example of isNotEmpty
```

```
print("The length of the string is: ${str.length}"); //Example of Length
```

Methods Of String

toLowerCase(): Converts all characters in this string to lowercase.

toUpperCase(): Converts all characters in this string to uppercase.

trim(): Returns the string without any leading and trailing whitespace.

compareTo(): Compares this object to another.

replaceAll(): Replaces all substrings that match the specified pattern with a given value.

split(): Splits the string at matches of the specified delimiter and returns a list of substrings.

toString(): Returns a string representation of this object.

substring(): Returns the text from any position you want.

codeUnitAt(): Returns the 16-bit UTF-16 code unit at the given index.

Syntax:

```
// for uppercase and lowercase variables
String address1 = " florida"; // Here F is capital
String address2 = "TEXAS"; // Here T and A are capital
// for trim variables
String item1 = "Apple";
String item2 = "Ant";
// for replaceAll variables
String text = "I am a good boy I like milk. Doctor says milk is good for health.";
String newText = text.replaceAll("milk", "water");
// for split variables
String allNames = "Ram, Hari, Shyam, Gopal";
List<String> listNames = allNames.split(",");
// for toString variables
int number = 20;
String result = number.toString();
// for substring variables
String text = "I love computer";
// for reverse variable
String input = "Hello";

print("Address 1 in uppercase: ${address1.toUpperCase()}"); //Address 1 in uppercase: FLORIDA
print("Address 2 in lowercase: ${address2.toLowerCase()}"); //Address 1 in lowercase: Texas

print("Result of address1 trim is ${address1.trim()}"); //Result of address1 trim is floarida
print("Comparing item 1 with item 2: ${item1.compareTo(item2)}"); //Comparing item 1 with item 2: 1

print("Original Text: $text"); //Original Text: I am a good boy I like milk. Doctor says milk is good for health.
```

```
print("Replaced Text: $newText"); //Replaced Text: I am a good boy I like water. Doctor says water is good for health.
```

```
print("Value of listName is $listNames"); //Value of listName is [Ram, Hari, Shyam, Gopal]
print("List name at 0 index ${listNames[0]}"); //List name at 0 index Ram
print("List name at 1 index ${listNames[1]}"); //List name at 1 index Hari
```

```
print("Type of number is ${number.runtimeType}"); //Type of number is int
print("Type of result is ${result.runtimeType}"); //Type of result is String
```

```
print("Print only computer: ${text.substring(7)}"); //from index 6 to the last index. //Print only computer:
computer
```

```
print("Print only love: ${text.substring(2,6)}"); //from index 2 to the 6th index. //Print only love: love
```

```
print("$input Reverse is ${input.split('').reversed.join()}"); //Hello Reverse is olleH
```

Conditions in Dart

When you write a computer program, you need to be able to tell the computer what to do in different situations. With conditions, you can control the flow of the dart program. Suppose you need to execute a specific code when a particular situation is true. In that case, you can use conditions in Dart.

Types Of Condition

If Condition

If-Else Condition

If-Else-If Condition

Switch case

1. If Condition

The easy and most common way of controlling the flow of a program is through the use of an if statement.

Syntax:

```
if(condition) {
    Statement 1;
    Statement 2;
    .
    .
    Statement n;
}
```


Eg:

```
if(age >= 18){  
    print("You are voter.");  
}
```

2. If-Else Condition

If the result of the condition is true, then the body of the if-condition is executed. Otherwise, the body of the else-condition is executed.

Syntax:

```
if(condition){  
    statements;  
}else{  
    statements;  
}
```

Eg:

```
int age = 12;  
if(age >= 18){  
    print("You are voter.");  
}else{  
    print("You are not voter.");  
}
```

3. Condition Based On Boolean Value

We can assign a Boolean value into a variable and we can use the condition by using that variable name.

Eg:

```
bool isMarried = false;  
if(isMarried){  
    print("You are married.");  
}else{  
    print("You are single.");  
}
```

4. If-Else-If Condition

When you have multiple if conditions, then you can use if-else-if.

Syntax:

```
if(condition1){
```

```

statements1;
}else if(condition2){
statements2;
}else if(condition3){
statements3;
}
.
.
else(conditionN){
statementsN;
}

```

Eg:

```
int noOfMonth = 5;
```

```

// Check the no of month
if (noOfMonth == 1) {
    print("The month is jan");
} else if (noOfMonth == 2) {
    print("The month is feb");
} else if (noOfMonth == 3) {
    print("The month is march");
} else if (noOfMonth == 4) {
    print("The month is april");
} else if (noOfMonth == 5) {
    print("The month is may");
} else if (noOfMonth == 6) {
    print("The month is june");
} else if (noOfMonth == 7) {
    print("The month is july");
} else if (noOfMonth == 8) {
    print("The month is aug");
} else if (noOfMonth == 9) {
    print("The month is sep");
} else if (noOfMonth == 10) {
    print("The month is oct");
} else if (noOfMonth == 11) {
    print("The month is nov");
} else if (noOfMonth == 12) {
    print("The month is dec");
} else {
    print("Invalid option given.");
}

```

the output will be "May". If the noOfMonth is not a number from 1 - 12 it prints Invalid option given.

Okay, We can learn with a problem first,

Find Greatest Number Among 3 Numbers

Code:

```
void main(){
    int num1 = 1200;
    int num2 = 1000;
    int num3 = 150;

    if(num1 > num2 && num1 > num3){
        print("Num 1 is greater: i.e $num1");
    }
    if(num2 > num1 && num2 > num3){
        print("Num2 is greater: i.e $num2");
    }
    if(num3 > num1 && num3 > num2){
        print("Num3 is greater: i.e $num3");
    }
}
```

output : Num 1 is greater: i.e 1200

5. Assert

While coding, it is hard to find errors in big projects, so dart provides an assert method to check for the error. It takes conditions as an argument. If the condition is true, nothing happens. If a condition is false, it will raise an error.

Syntax:

```
assert(condition);
// or
assert(condition, "Your custom message");
```

Eg:

```
var age = 22;
assert(age!=22);
```

output: Uncaught Error: Assertion failed

Eg2:

```
var age = 22;
assert(age!=22, "Age must be 22");
```

output: Uncaught Error: Assertion failed: "Age must be 22"

We have to Run File In Assert Mode, Use this command to run the dart file, which enables assert. If you don't use this, you will not be able to see the issue. You can use this command below if you are running a dart file from the computer.

`"dart --enable-asserts file_name.dart"`

The `assert(condition)` method only runs in development mode. It will throw an exception only when the condition is false. If the condition is true, nothing happens. Production code ignores it.

6. Switch Case in Dart

A Switch case is used to execute the code block based on the condition, Same as if-else-if condition. You can use a Switch case as an alternative to the if-else-if condition.

Syntax:

```
switch(expression) {  
  case value1:  
    // statements  
    break;  
  case value2:  
    // statements  
    break;  
  case value3:  
    // statements  
    break;  
  default:  
    // default statements  
}
```

How does switch-case statement work in dart

Evaluate Once: The switch checks an expression just once.

Check Cases: It compares the result with each case value.

Run Matched Case: If it finds a match, it runs that case's code.

Break to Exit: Use break to leave the switch after a case runs.

Default Case: If no case matches, the default code runs (if you have one).

If-else-If

```
var dayOfWeek = 5;  
if (dayOfWeek == 1) {  
  print("Day is Sunday.");  
}
```

Switch Statement

```
var dayOfWeek = 5;  
switch (dayOfWeek) {  
  case 1:
```

```

    }
    else if (dayOfWeek == 2) {
        print("Day is Monday.");
    }
    else if (dayOfWeek == 3) {
        print("Day is Tuesday.");
    }
    else if (dayOfWeek == 4) {
        print("Day is Wednesday.");
    }
    else if (dayOfWeek == 5) {
        print("Day is Thursday.");
    }
    else if (dayOfWeek == 6) {
        print("Day is Friday.");
    }
    else if (dayOfWeek == 7) {
        print("Day is Saturday.");
    }else{
        print("Invalid Weekday.");
    }

    print("Day is Sunday.");
    break;
case 2:
    print("Day is Monday.");
    break;
case 3:
    print("Day is Tuesday.");
    break;
case 4:
    print("Day is Wednesday.");
    break;
case 5:
    print("Day is Thursday.");
    break;
case 6:
    print("Day is Friday.");
    break;
case 7:
    print("Day is Saturday.");
    break;
default:
    print("Invalid Weekday.");
    break;
}

```

By the use of Switch, The syntax of switch statements is cleaner and much easier to read and write.

7. Ternary Operator

The ternary operator is like an if-else statement. This is a one-liner replacement for the if-else statement. It is used to write a conditional expression, where based on the result of a boolean condition, one of the two values is selected.

Syntax:

condition ? expressionTrue : expressionFalse;

Ternary Operator Vs If Else

This program finds the greatest number between two numbers using if else.

if Else

```
void main() {  
  int num1 = 10;  
  int num2 = 15;  
  int max = 0;  
  if(num1 > num2){  
    max = num1;  
  }else {  
    max = num2;  
  }  
  print("The greatest number is $max");  
}
```

Ternary operator

```
void main() {  
  int num1 = 10;  
  int num2 = 15;  
  int max = (num1 > num2) ? num1 : num2;  
  print("The greatest number is $max");  
}
```

Dart Loops

In Programming, loops are used to repeat a block of code until certain conditions are not completed.

Here is a difference between Print Your Name 10 Times Without Using Loop and With loop.

Print Your Name 10 Times Without Using Loop

```
print("John");  
print("John");  
print("John");  
print("John");  
print("John");  
print("John");  
print("John");  
print("John");  
print("John");  
print("John");
```

Print Your Name 10 Times Using Loop

```
for (int i = 0; i < 10; i++) {  
  print("John Doe");  
}
```

There are different types of loop in Dart. They are:

For Loop

For Each Loop

While Loop

Do While Loop

1. For Loop

This is the most common type of loop. You can use for loop to run a code block multiple times according to the condition. The syntax of for loop is:

Syntax:

```
for(initialization; condition; increment/decrement){  
    statements;  
}
```

Eg:

To Print 1 To 10 Using For Loop

```
for (int i = 1; i <= 10; i++) {  
    print(i);  
}
```

output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

2. For Each Loop

The for each loop iterates over all list elements or variables. It is useful when you want to loop through a list/collection.

Syntax:

```
collection.forEach(void f(value));
```

Eg:

```
List<String> footballplayers=["Ronaldo","Messi","Neymar","Hazard"];  
footballplayers.forEach( (names)=>print(names));
```

output:

```
Ronaldo  
Messi  
Neymar  
Hazard
```

3. While Loop

In while loop, the loop's body will run until and unless the condition is true. You must write conditions first before statements. This loop checks conditions on every iteration. If the condition is true, the code inside {} is executed, if the condition is false, then the loop stops.

Syntax:

```
while(condition){  
    //statement(s);  
    // Increment (++) or Decrement (--) Operation;  
}
```

Eg:

```
int i = 1;  
while (i <= 10) {  
    print(i);  
    i++;  
}
```

output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

4. Do While Loop

Do while loop is used to run a block of code multiple times. The loop's body will be executed first, and then the condition is tested.

Syntax:

```
do{  
    statement1;  
    statement2;  
    .  
    .  
    statementN;  
} while(condition);
```


Eg:

```
int i = 1;
do {
    print(i);
    i++;
} while (i <= 10);
```

Dart Break and Continue

While working on loops, we need to skip some elements or terminate the loop immediately without checking the condition. In such a situation, you can use the break and continue statement.

Break Statement

Sometimes you will need to break out of the loop immediately without checking the condition. You can do this using break statement and use "break" to stop the execution of program.

Eg:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    print(i);
}
```

Continue Statement

Sometimes you will need to skip an iteration for a specific condition. You can do this using continue statement and use "continue" to skip the element of program.

Eg:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        continue;
    }
    print(i);
}
```

Exception Handling in Dart

An exception is an error that occurs at runtime during program execution. When the exception occurs, the flow of the program is interrupted, and the program terminates abnormally.

There is a high chance of crashing or terminating the program when an exception occurs. Therefore, to save your program from crashing, you need to catch the exception.

Syntax:

```
try {  
  // Your Code Here  
}  
catch(ex){  
  // Exception here  
}
```

Try: To write the logical code that creates exceptions in the try block.

Catch: When you are uncertain about what kind of exception a program produces, then a catch block is used. It is written with a try block to catch the general exception.

Eg:

```
int a = 18;  
int b = 0;  
int res;  
  
try {  
  res = a ~/ b;  
  print("Result is $res");  
}  
// It returns the built-in exception related to the occurring exception  
catch(ex) {  
  print(ex);  
}
```

Function In Dart

Functions are the block of code that performs a specific task. They are created when some statements are repeatedly occurring in the program. The function helps reusability of the code in the program.

Function Advantages

1. Avoid Code Repetition
2. Easy to divide the complex program into smaller parts
3. Helps to write a clean code

Syntax:

```
returntype functionName(parameter1,parameter2, ...){  
    // function body  
}
```

return type: The return type tells you what type of value a function will give back, like void, String, int, or double. If a function doesn't return anything, you use void.

functionName: You can name functions almost anything, but use lowerCamelCase, like this: void printName().

parameters: Parameters are the inputs for a function, written inside the brackets (). Use lowerCamelCase for naming your parameters.

Eg:

Function That Prints Name

// writing function outside main function.

```
void printName(){  
    print("My name is Raj Sharma. I am from function.");  
}
```

// this is our main function.

```
void main(){  
    printName();  
}
```

output: My name is Raj Sharma. I am from function.

Eg2:

Function To Find Sum of Two Numbers

```
void add(int num1, int num2){  
    int sum = num1 + num2;  
    print("The sum is $sum");  
}
```

```
void main(){  
    add(10, 20);  
}
```

output: The sum is 30

Types Of Function

Functions are the block of code that performs a specific task. Here are different types of functions:

Function without parameter and without return value

Function with Parameter without return value

Function without Parameter with return value

Function with Parameter with return value

1. Function without parameter and without return value

In this function, you do not pass any parameter and expect no return type.

Eg:

```
void main() {  
    printName();  
}  
void printName() {  
    print("My name is John Doe.");  
}  
output: My name is John Doe.
```

2. Function with Parameter without return value

In this function, you do pass the parameter and expect no return type.

Eg:

```
void main() {  
    print("Function with Parameter without return value");  
    printPrimeMinisterName();  
}
```

```
void printPrimeMinisterName() {  
    print("John Doe.");  
}
```

output:

Function with Parameter without return value
John Doe.

3. Function without Parameter with return value

In this function, you do not pass any parameter but expect return type.

Eg:

```
void main() {  
  // Function With No Parameter & Return Type  
  String name = primeMinisterName();  
  print("The Name from function is $name.");  
}  
String primeMinisterName() {  
  return "John Doe";  
}
```

output: The name from function is John Doe

4. Function with Parameter with return value

In this function, you do pass the parameter and also expect return type.

Eg:

```
// this function add two numbers  
int add(int a, int b) {  
  int sum = a + b;  
  return sum;  
}
```

```
void main() {  
  int num1 = 10;  
  int num2 = 20;  
  
  int total = add(num1, num2);  
  print("The sum is $total.");  
}
```

output: The sum is 30.

We've encountered parameters in many contexts. Do you know what a parameter is?
Let's discuss it!

Parameter In Dart

The parameter is the process of passing values to the function. The values passed to the function must match the number of parameters defined. A function can have any number of parameters.

Syntax:

```
void add(int a, int b) {  
}
```

Types of Parameters

Positional Parameter

Named Parameter

Optional Parameter

1. Positional Parameter

With positional parameters, you need to provide the arguments in the same order as you defined the parameters in the function. If you call the function with the arguments in the wrong order, you'll get the wrong result.

Eg:

```
void printInfo(String name, String gender) {  
    print("Hello $name your gender is $gender.");  
}
```

```
void main() {  
    // passing values in wrong order  
    printInfo("Male", "John");  
  
    // passing values in correct order  
    printInfo("John", "Male");  
}
```

output:

Hello Male your gender is John.

Hello John your gender is Male.

2. Named Parameters

Dart allows you to use named parameters to clarify the parameter's meaning in function calls. Curly braces {} are used to specify named parameters.

Eg:

```
void printInfo({String? name, String? gender}) {  
    print("Hello $name your gender is $gender.");  
}
```

```

void main() {
  // you can pass values in any order in named parameters.
  printInfo(gender: "Male", name: "John");
  printInfo(name: "Sita", gender: "Female");
  printInfo(name: "Reecha", gender: "Female");
  printInfo(name: "Reecha", gender: "Female");
  printInfo(name: "Harry", gender: "Male");
  printInfo(gender: "Male", name: "Santa");
}

```

output:

```

Hello John your gender is Male.
Hello Sita your gender is Female.
Hello Reecha your gender is Female.
Hello Reecha your gender is Female.
Hello Harry your gender is Male.
Hello Santa your gender is Male.

```

3. Optional Parameter

Dart allows you to use optional parameters to make the parameter optional in function calls. Square braces [] are used to specify optional parameters.

Eg:

```

void printInfo(String name, String gender, [String? title]) {
  print("Hello $title $name your gender is $gender.");
}

```

```

void main() {
  printInfo("John", "Male");
  printInfo("John", "Male", "Mr.");
  printInfo("Kavya", "Female", "Ms.");
}

```

output:

```

Hello John your gender is Male.
Hello Mr. John your gender is Male.
Hello Ms. Kavya your gender is Female.

```

Anonymous Function In Dart

If you remove the return type and the function name, the function is called anonymous function.

Syntax:

```

(parameterList){
  // statements
}

```

Eg:

```
void main() {  
  // Anonymous function  
  var cube = (int number) {  
    return number * number * number;  
  };  
  
  print("The cube of 2 is ${cube(2)}");  
  print("The cube of 3 is ${cube(3)}");  
}
```

output:

The cube of 2 is 8
The cube of 3 is 27

Arrow Function In Dart

Dart has a special syntax for the function body, which is only one line. The arrow function is represented by => symbol. It is a shorthand syntax for any function that has only one expression.

Syntax:

returnType functionName(parameters...) => expression;

The arrow function is used to make your code short. => expr syntax is a shorthand for { return expr; }.

Eg: Simple Interest Without Arrow Function

```
// function that calculate interest  
double calculateInterest(double principal, double  
rate, double time) {  
  double interest = principal * rate * time / 100;  
  return interest;  
}  
  
void main() {  
  double principal = 5000;  
  double time = 3;  
  double rate = 3;  
  
  double result = calculateInterest(principal, rate,  
time);  
  print("The simple interest is $result.");  
}
```

Eg: Calculate Simple Interest With Arrow Function

```
// arrow function that calculate interest  
double calculateInterest(double principal, double  
rate, double time) =>  
  principal * rate * time / 100;  
  
void main() {  
  double principal = 5000;  
  double time = 3;  
  double rate = 3;  
  
  double result = calculateInterest(principal, rate,  
time);  
  print("The simple interest is $result.");  
}  
output: Simple interest is 450.0
```


output: Simple interest is 450.0

Scope in Dart

Scope is very important in Dart. It determines where a variable or function is accessible within your code. Understanding scope helps prevent naming conflicts and ensures that you can manage variables effectively, whether they're local to a function or available throughout a class or the entire program.

Types of Scope in Dart

Method Scope

Global Scope

1. Method Scope

If you created variables inside the method, you can use them inside the method block but not outside the method block.

Eg:

```
void main() {  
  String text = "I am text inside main. Anyone can't access me.";  
  print(text);  
}
```

output: I am text inside main. Anyone can't access me.

In this program, text is a String type where you can access and print method only inside the main function but not outside the main function.

2. Global Scope

You can define a variable in the global scope to use the variable anywhere in your program.

Eg:

```
String global = "I am Global. Anyone can access me.";  
void main() {  
  print(global);  
}
```

output:I am Global. Anyone can access me.

In this program, the variable named global is a top-level variable; you can access it anywhere in the program.

Math In Dart

Math helps you to perform mathematical calculations efficiently. With dart math, you can generate random numbers, find square root, find power of number, or round specific numbers. To use math in dart, you must import 'dart:math';.

You can Generate Random Numbers In Dart using math function,

Eg:

generate random numbers from 0 - 9 and also 1 to 10.

```
import 'dart:math';
```

```
void main()
```

```
{
```

```
  Random random = new Random();
```

```
  int randomNumber = random.nextInt(10); // from 0 to 9 included
```

```
  print("Generated Random Number Between 0 to 9: $randomNumber");
```

```
  int randomNumber2 = random.nextInt(10)+1; // from 1 to 10 included
```

```
  print("Generated Random Number Between 1 to 10: $randomNumber2");
```

```
}
```

In this program, random.nextInt(10) function is used to generate a random number between 0 and 9 in which the value is stored in a variable randomNumber.

The random.nextInt(10)+1 function is used to generate a random number between 1 to 10 in which the value is stored in a variable randomNumber2.

1. Generate Random Number Between Any Number

Syntax:

```
min + Random().nextInt((max + 1) - min);
```

Eg: random numbers between 10 to 20.

```
import 'dart:math';
```

```
void main()
```

```
{
```

```
  int min = 10;
```

```

int max = 20;

int randomnum = min + Random().nextInt((max + 1) - min);

print("Generated Random number between $min and $max is: $randomnum");
}

```

2. Random Boolean And Double Value

Syntax:

```

Random().nextBool(); // return true or false
Random().nextDouble(); // return 0.0 to 1.0

```

Eg: generate random and boolean value.

```

import 'dart:math';
void main()
{
double randomDouble = Random().nextDouble();
bool randomBool = Random().nextBool();

print("Generated Random double value is: $randomDouble");
print("Generated Random bool value is: $randomBool");
}

```

Math Functions in Dart

You can use some useful math functions to perform your daily task with dart programming.

Function Name	Output	Description
pow(10,2)	100	10 to the power 2 is 10*10
max(10,2)	10	Maximum number is 10
min(10,2)	2	Minimum number is 2
sqrt(25)	5	Square root of 25 is 5

Eg:

```

import 'dart:math';
void main()
{
int num1 = 10;
int num2 = 2;

num powernum = pow(num1,num2);
}

```

```

num maxnum = max(num1,num2);
num minnum = min(num1,num2);
num squareroot = sqrt(25); // Square root of 25

print("Power is $powernum");
print("Maximum is $maxnum");
print("Minimum is $minnum");
print("Square root is $squareroot");
}
output:
Power is 100
Maximum is 10
Minimum is 2
Square root is 5.0

```

Collections in Dart

In Dart, "collections" refer to the built-in data structures used to store and manipulate groups of objects.

Dart provides several types of collections, primarily:

- List
- Set
- Map
- Where

1. List

An ordered collection of items, which can be of any type. Lists allow duplicate entries and have a zero-based index.

syntax:

```

// Integer List
List<int> ages = [10, 30, 23];
// String List
List<String> names = ["Raj", "John", "Rocky"];
// Mixed List
var mixed = [10, "John", 18.8];

```

List also contain some types,

- Fixed Length List
- Growable List [Mostly Used]

- **Fixed Length List**

The fixed-length Lists are defined with the specified length. You cannot change the size at runtime. This will create a List of 5 integers with the value 0.

Eg:

```
void main() {  
    var list = List<int>.filled(5,0);  
    print(list);  
}  
output: [0, 0, 0, 0, 0]
```

- **Growable List**

A List defined without a specified length is called Growable List. The length of the growable List can be changed at runtime.

Eg:

```
void main() {  
    var list1 = [210,21,22,33,44,55];  
    print(list1);  
}  
output: [210, 21, 22, 33, 44, 55]
```

Access Item Of List

You can access the List item by index. Remember that the List index always starts with 0.

Eg:

```
void main() {  
    var list = [210, 21, 22, 33, 44, 55];  
  
    print(list[0]);  
    print(list[1]);  
    print(list[2]);  
    print(list[3]);  
    print(list[4]);  
    print(list[5]);  
}  
output:  
210  
21  
22  
33
```

44
55

1. Get Index By Value

You can also get the index by value.

Eg:

```
void main() {  
    var list = [210, 21, 22, 33, 44, 55];  
  
    print(list.indexOf(22));  
    print(list.indexOf(33));  
}  
output:  
2  
3
```

2. Find The Length Of The List

You can find the length of List by using .length property. List index starts with 0 and length always starts with 1.

Eg:

```
void main(){  
    List<String> names = ["Raj", "John", "Rocky"];  
    print(names.length);  
}  
output: 3
```

3. Changing Values Of List

You can also change the value of List. You can do it by listName[index]=value;.

Eg:

```
void main(){  
    List<String> names = ["Raj", "John", "Rocky"];  
    names[1] = "Bill";  
    names[2] = "Elon";  
    print(names);  
}  
output: [Raj, Bill, Elon]
```

4. Mutable And Immutable List

A mutable List means they can change after the declaration, and an immutable List means they can't change after the declaration.

Eg1:

```
List<String> names = ["Raj", "John", "Rocky"]; // Mutable List
names[1] = "Bill"; // possible to change values
names[2] = "Elon"; // possible to change values
```

Eg2:

```
const List<String> names = ["Raj", "John", "Rocky"]; // Mutable List
names[1] = "Bill"; // not possible values
names[2] = "Elon"; // not possible to change values
```

List Properties In Dart

first: It returns the first element in the List.

last: It returns the last element in the List.

isEmpty: It returns true if the List is empty and false if the List is not empty.

isNotEmpty: It returns true if the List is not empty and false if the List is empty.

length: It returns the length of the List.

reversed: It returns a List in reverse order.

single: It is used to check if the List has only one element and returns it.

1. Access First And Last Elements Of List

You can access the first and last elements in the List by,

Eg:

```
void main() {
  List<String> drinks = ["water", "juice", "milk", "coke"];
  print("First element of the List is: ${drinks.first}");
  print("Last element of the List is: ${drinks.last}");
}
```

output:

First element of the List is: water

Last element of the List is: coke

2. Check The List Is Empty Or Not

You can also check if a List contains any elements inside it or not. It will give results either in true or in false.

Eg:

```
void main() {  
    List<String> drinks = ["water", "juice", "milk", "coke"];  
    List<int> ages = [];  
    print("Is drinks Empty: "+drinks.isEmpty.toString());  
    print("Is drinks not Empty: "+drinks.isNotEmpty.toString());  
    print("Is ages Empty: "+ages.isEmpty.toString());  
    print("Is ages not Empty: "+ages.isNotEmpty.toString());  
}
```

}

output:

Is drinks Empty: false
Is drinks not Empty: true
Is ages Empty: true
Is ages not Empty: false

3. Reverse List In Dart

You can easily reverse List by using .reversed properties.

Eg:

```
void main() {  
    List<String> drinks = ["water", "juice", "milk", "coke"];  
    print("List in reverse: ${drinks.reversed}");  
}
```

}

output: List in reverse: (coke, milk, juice, water)

Adding Item To List

Dart provides four methods to insert the elements into the Lists.

Method	Description
add() addAll()	Add one element at a time and returns the modified List object. Insert the multiple values to the given List, and each value is separated by the commas and enclosed with a square bracket ([]).
insert() insertAll()	Provides the facility to insert an element at a specified index position. Insert the multiple value at the specified index position.

Eg:

```
void main() {  
  var evenList = [2, 4, 6, 8, 10];  
  print(evenList);  
  evenList.add(12); // add item to list  
  print(evenList);  
  evenList.addAll([13, 14, 15, 16]); // add list to list  
  print(evenList);  
  myList.insert(2, 15); // insert 15 to 2nd index of list  
  print(myList);  
}
```

output:

[2, 4, 6, 8, 10]

[2, 4, 6, 8, 10, 12]

[2, 4, 6, 8, 10, 12, 13, 14, 15, 16]

[2, 4, 15, 6, 8, 10, 12, 13, 14, 15, 16]

Replace Range Of List

You can also replace the range of the List.

Eg:

```
void main() {  
  var list = [10, 15, 20, 25, 30];  
  print("List before updation: ${list}");  
  list.replaceRange(0, 4, [5, 6, 7, 8]);  
  print("List after updation using replaceAll() function : ${list}");  
}
```

output:

List before updation: [10, 15, 20, 25, 30]

List after updation using replaceAll() function : [5, 6, 7, 8, 30]

Removing List Elements

There are some methods in dart to remove list elements

Method	Description
remove()	Removes one element at a time from the given List. Removes an element from the specified index position and returns it.
removeAt()	
removeLast()	

removeRange()	Remove the last element from the given List. Removes the item within the specified range.
---------------	--

Removing List Item From List

We can remove items from List using remove() method. It will remove the value that is available in the index of the list. So, we can pass values to remove the element in the list.

Eg:

```
void main() {
    var list = [10, 20, 30, 40, 50];
    print("List before removing element : ${list}");
    list.remove(30);
    print("List after removing element : ${list}");
}
```

output:

List before removing element : [10, 20, 30, 40, 50]

List after removing element : [10, 20, 40, 50]

Removing List Item From List

We can remove items from List using removeAt() method. It will also remove the value that is available in the index of the list. So, we can pass an index number to remove the element in the list.

Eg:

```
void main() {
    var list = [10, 11, 12, 13, 14];
    print("List before removing element : ${list}");
    list.removeAt(3);
    print("List after removing element : ${list}");
}
```

output:

List before removing element : [10, 11, 12, 13, 14]

List after removing element : [10, 11, 12, 14]

Removing Last Item From List

We can remove items from List using `removeLast()` method. It will also remove the last value of the list.

Eg:

```
void main() {  
  var list = [10, 20, 30, 40, 50];  
  print("List before removing element:${list}");  
  list.removeLast();  
  print("List after removing last element:${list}");  
}  
output:  
List before removing element:[10, 20, 30, 40, 50]  
List after removing last element:[10, 20, 30, 40]
```

Loops In List

You can use for loop, for each loop, or any other type of loop.

Eg:

```
void main() {  
  List<int> list = [10, 20, 30, 40, 50];  
  list.forEach((n) => print(n));  
}  
output:  
10  
20  
30  
40  
50
```

Multiply All Value By 2 Of All List

Eg:

```
void main() {  
  List<int> list = [10, 20, 30, 40, 50];  
  var douledList = list.map((n) => n * 2);  
  
  print((douledList));  
}  
output: (20, 40, 60, 80, 100)
```

Combine Two Or More List In Dart

You can combine two or more Lists in dart by using spread operator syntax.

Eg:

```
void main() {  
    List<String> names = ["Raj", "John", "Rocky"];  
    List<String> names2 = ["Mike", "Subash", "Mark"];  
  
    List<String> allNames = [...names, ...names2];  
    print(allNames);  
}  
output: [Raj, John, Rocky, Mike, Subash, Mark]
```

Conditions In List

You can also use conditions in List. Here sad = false so cart doesn't contain Beer in it.

Eg:

```
void main() {  
    bool sad = false;  
    var cart = ['milk', 'ghee', if (sad) 'Beer'];  
    print(cart);  
}  
output: [milk, ghee]
```

Where In List Dart

You can use where with List to filter specific items.

Eg:

```
void main(){  
    List<int> numbers = [2,4,6,8,10,11,12,13,14];  
  
    List<int> even = numbers.where((number)=> number.isEven).toList();  
    print(even);  
}  
output: [2, 4, 6, 8, 10, 12, 14]
```

2. Set

Set is a unique collection of items. You cannot store duplicate values in the Set. It is unordered, so it can be faster than lists while working with a large amount of data. Set is useful when you need to store unique values without considering the order of the input. It is represented by Curley Braces{ }.

Syntax: Set <variable_type> variable_name = {};

How To Create A Set In Dart

You can create a Set in Dart using the Set type annotation. Here Set<String> means only text is allowed in the Set.

Eg:

```
void main(){
  Set<String> fruits = {"Apple", "Orange", "Mango"};
  print(fruits);
}
output: {Apple, Orange, Mango}
```

Set Properties

first: To get first value of Set.

last: To get last value of Set.

isEmpty: Return true or false.

isNotEmpty: Return true or false.

length: It returns the length of the Set.

Eg:

```
// declaring fruits as Set
Set<String> fruits = {"Apple", "Orange", "Mango", "Banana"};

// using different properties of Set
print("First Value is ${fruits.first}");
print("Last Value is ${fruits.last}");
print("Is fruits empty? ${fruits.isEmpty}");
print("Is fruits not empty? ${fruits.isNotEmpty}");
print("The length of fruits is ${fruits.length}");
output:
First Value is Apple
Last Value is Banana
Is fruits empty? false
Is fruits not empty? true
The length of fruits is 4
```

Add & Remove Items In Set

Like lists, you can add or remove items in a Set. To add items use add() method and to remove use remove() method.

add(): Add one element to Set.

remove(): Removes one element from Set.

Eg:

```
void main(){
    Set<String> fruits = {"Apple", "Orange", "Mango"};

    fruits.add("Lemon");
    fruits.add("Grape");

    print("After Adding Lemon and Grape: $fruits");

    fruits.remove("Apple");
    print("After Removing Apple: $fruits");
}
output:
After Adding Lemon and Grape: {Apple, Orange, Mango, Lemon, Grape}
After Removing Apple: {Orange, Mango, Lemon, Grape}
```

Adding Multiple Elements in Sets

You can use **addAll()** method to add multiple elements from the list to Set.

addAll(): Insert the multiple values to the given Set.

Eg:

```
void main(){
    Set<int> numbers = {10, 20, 30};
    numbers.addAll([40,50]);
    print("After adding 40 and 50: $numbers");
}
output: After adding 40 and 50: {10, 20, 30, 40, 50}
```

Set Methods

clear(): Removes all elements from the Set.

difference(): Creates a new Set with the elements of this that are not in other.

elementAt(): Returns the index value of element.

intersection(): Find common elements in two sets.

Eg:

```
void main() {
    Set<String> fruits1 = {"Apple", "Orange", "Mango"};
    Set<String> fruits2 = {"Apple", "Grapes", "Banana"};
```

```

Set<String> days = {"Sunday", "Monday", "Tuesday"};

final differenceSet = fruits1.difference(fruits2);
fruits.clear();
print(fruits);
print(differenceSet);
print(days.elementAt(2));
final intersectionSet = fruits1.intersection(fruits2);
print(intersectionSet);
}
output:
{}
{Orange, Mango}
Tuesday
{Apple}

```

3. Map

In a Map, data is stored as keys and values. In Map, each key must be unique. They are similar to HashMaps and Dictionaries in other languages.

How to create a Map in dart

Eg:

```

void main(){
Map<String, String> countryCapital = {
  'USA': 'Washington, D.C.',
  'India': 'New Delhi',
  'China': 'Beijing'
};
  print(countryCapital);
}
output: {USA: Washington, D.C., India: New Delhi, China: Beijing}

```

Access Value From Key

You can find the value of Map from its key.

Eg:

```

void main(){
Map<String, String> countryCapital = {
  'USA': 'Washington, D.C.',
  'India': 'New Delhi',
  'China': 'Beijing'
};
  print(countryCapital["USA"]);
}

```

```
}
```

output: Washington, D.C.

Map Properties In Dart

keys: To get all keys.

values: To get all values.

isEmpty: Return true or false.

isNotEmpty: Return true or false.

length: It returns the length of the Map.

Eg:

```
void main() {  
  
    Map<String, double> expenses = {  
        'sun': 3000.0,  
        'mon': 3000.0,  
        'tue': 3234.0,  
    },  
    print("All keys of Map: ${expenses.keys}");  
    print("All values of Map: ${expenses.values}");  
    print("Is Map empty: ${expenses.isEmpty}");  
    print("Is Map not empty: ${expenses.isNotEmpty}");  
    print("Length of map is: ${expenses.length}");  
}
```

output:

All keys of Map: (sun, mon, tue)
All values of Map: (3000, 3000, 3234)
Is Map empty: false
Is Map empty: true
Length of map is: 3

- **Adding Element To Map**

If you want to add an element to the existing Map.

Eg:

```
void main(){  
    Map<String, String> countryCapital = {  
        'USA': 'Washington, D.C.',  
        'India': 'New Delhi',  
        'China': 'Beijing'  
    };  
    // Adding New Item
```



```
countryCapital['Japan'] = 'Tokio';
print(countryCapital);
}
```

output: {USA: Washington, D.C., India: New Delhi, China: Beijing, Japan: Tokio}

- **Updating An Element Of Map**

If you want to update an element of the existing Map.

Eg:

```
void main(){
Map<String, String> countryCapital = {
    'USA': 'Nothing',
    'India': 'New Delhi',
    'China': 'Beijing'
};
// Updating Item
countryCapital['USA'] = 'Washington, D.C.';
print(countryCapital);
}
```

output: {USA: Washington, D.C., India: New Delhi, China: Beijing}

Map Methods

keys.toList(): Convert all Maps keys to List.

values.toList(): Convert all Maps values to List.

containsKey('key'): Return true or false.

containsValue('value'): Return true or false.

clear(): Removes all elements from the Map.

removeWhere(): Removes all elements from the Map if condition is valid.

Eg:

```
void main() {

    Map<String, double> expenses = {
        'sun': 3000.0,
        'mon': 3000.0,
        'tue': 3234.0,
    };
    // Without List
    print("All keys of Map: ${expenses.keys}");
    print("All values of Map: ${expenses.values}");

    // With List
    print("All keys of Map with List: ${expenses.keys.toList()}");
}
```

```

print("All values of Map with List: ${expenses.values.toList()}");

// For Keys
print("Does Map contain key sun: ${expenses.containsKey("sun")}");
print("Does Map contain key abc: ${expenses.containsKey("abc")}");

// For Values
print("Does Map contain value 3000.0: ${expenses.containsValue(3000.0)}");
print("Does Map contain value 100.0: ${expenses.containsValue(100.0)}");

}
output:
All keys of Map: (sun, mon, tue)
All values of Map: (3000, 3000, 3234)
All keys of Map with List: [sun, mon, tue]
All values of Map with List: [3000, 3000, 3234]
Does Map contain key sun: true
Does Map contain key abc: false
Does Map contain value 3000.0: true
Does Map contain value 100.0: false

```

- **Removing Items From Map**

You can remove an element of the existing Map, Use `mapname.remove("key")`.

Eg:

```

void main(){
Map<String, String> countryCapital = {
    'USA': 'Nothing',
    'India': 'New Delhi',
    'China': 'Beijing'
};

countryCapital.remove("USA");
print(countryCapital);
}
output: {India: New Delhi, China: Beijing}

```

- **Looping Over Element Of Map**

You can use any loop in Map to print all keys/values or to perform operations in its keys and values.

Eg:

```

void main(){

```

```

Map<String, dynamic> book = {
  'title': 'Misson Mangal',
  'author': 'Kuber Singh',
  'page': 233
};

// Loop Through Map
for(MapEntry book in book.entries){
  print('Key is ${book.key}, value ${book.value}');
}
}
output:
Key is title, value Misson Mangal
Key is author, value Kuber Singh
Key is page, value 233

```

- **Looping In Map Using For Each**

You can loop through map using for each loop also

Eg:

```

void main(){

  Map<String, dynamic> book = {
    'title': 'Misson Mangal',
    'author': 'Kuber Singh',
    'page': 233
  };

  // Loop Through For Each
  book.forEach((key,value)=> print('Key is $key and value is $value'));

}
output:
Key is title and value is Misson Mangal
Key is author and value is Kuber Singh
Key is page and value is 233

```

- **Remove Where In Dart Map**

You can also remove elements from a map using their index.

Eg:

```

void main() {

```

```

Map<String, double> mathMarks = {
  "ram": 30,
  "mark": 32,
  "harry": 88,
  "raj": 69,
  "john": 15,
};
mathMarks.removeWhere((key, value) => value < 32);
print(mathMarks);
}
output: {mark: 32.0, harry: 88.0, raj: 69.0}

```

4. Where

In Dart, you can use the where method with lists, sets, and maps to filter specific items. This method returns a new collection containing all elements that meet the specified condition. It's commonly referred to as the 'Where Filter'.

Syntax:

```

Iterable<E> where(
  bool test(
    E element
  )
)

```

Eg: Filter Only Odd Number From List

```

void main() {
  List<int> numbers = [2, 4, 6, 8, 10, 11, 12, 13, 14];

  List<int> oddNumbers = numbers.where((number) => number.isOdd).toList();
  print(oddNumbers);
}
output: [11, 13]

```

OOPs in Dart

Object-oriented programming (OOP) is a programming method that uses objects and their interactions to design and program applications. It is one of the most popular programming paradigms and is used in many programming languages, such as Dart, Java, C++, Python, etc.

In OOP, an object can be anything, such as a person, a bank account, a car, or a house. Each object has its attributes (or properties) and behavior (or methods). The main purpose of OOP is to break complex problems into smaller objects.

Advantages

Easy to understand and use.

Increases reusability and decreases complexity.

The productivity of programmers increases.

Makes the code easier to maintain, modify and debug.

Promotes teamwork and collaboration.

Reduces the repetition of code.

Features Of OOP

Class

Object

Encapsulation

Inheritance

Polymorphism

Abstraction

1. Class in Dart

In object-oriented programming, a class is a blueprint for creating objects. A class defines the properties and methods that an object will have.

Eg: a class called Dog might have properties like breed, color and methods like bark, run.

Declaring Class in Dart

Syntax:

```
class ClassName {  
  // properties or fields  
  // methods or functions  
}
```

class: this keyword is used for defining the class.

ClassName: name of the class and must start with capital letter.

Body of the class consists of properties and functions.

Properties are used to store the data. It is also known as fields or attributes. Functions are used to perform the operations. It is also known as methods.

Eg:

```
class Animal {  
    String? name;  
    int? numberOfLegs;  
    int? lifeSpan;  
  
    void display() {  
        print("Animal name: $name.");  
        print("Number of Legs: $numberOfLegs.");  
        print("Life Span: $lifeSpan.");  
    }  
}
```

2. Object In Dart

In object-oriented programming, an object is a self-contained unit of code and data. Objects are created from templates called classes. An object is made up of properties(variables) and methods(functions). An object is an instance of a class.

Instantiation

In object-oriented programming, instantiation is the process of creating an instance of a class. In other words, you can say that instantiation is the process of creating an object of a class.

Declaring Object In Dart

Once you have created a class, you can declare the object.

Syntax:

```
ClassName objectName = ClassName();
```

Eg:

```
class Bicycle {  
    String? color;  
    int? size;  
    int? currentSpeed;  
  
    void changeGear(int newValue) {  
        currentSpeed = newValue;  
    }  
}
```

```

    }

    void display() {
        print("Color: $color");
        print("Size: $size");
        print("Current Speed: $currentSpeed");
    }
}

void main(){
    Bicycle bicycle = Bicycle(); // Here bicycle is object of class Bicycle.
    bicycle.color = "Red";
    bicycle.size = 26;
    bicycle.currentSpeed = 0;
    bicycle.changeGear(5);
    bicycle.display();
}

```

output:

```

Color: Red
Size: 26
Current Speed: 5

```

3. Constructor In Dart

A constructor is a special method used to initialize an object. It is called automatically when an object is created, and it can be used to set the initial values for the object's properties.

The constructor's name should be the same as the class name.
 Constructor doesn't have any return type.

Syntax:

```

Person person = Person("John", 30);

```

Without a Constructor, you need to set the values of the properties manually.

Eg:

```

Person person = Person();
person.name = "John";
person.age = 30;

```

else, You dont need to set the value you can directly pass,

Eg: `Person person = Person("John", 30);`.

In dart, There are four types of Constructor

Default Constructor

Parameterized Constructor

Named Constructor

Constant Constructor

a. Default Constructor

The constructor which is automatically created by the dart compiler if you don't create a constructor is called a default constructor. A default constructor has no parameters. A default constructor is declared using the class name followed by parentheses ().

Eg:

```
class Laptop {  
  String? brand;  
  int? price;  
  
  // Constructor  
  Laptop() {  
    print("This is a default constructor");  
  }  
}  
  
void main() {  
  // Here laptop is object of class Laptop.  
  Laptop laptop = Laptop();  
}  
output: This is a default constructor
```

b. Parameterized Constructor

Parameterized constructor is used to initialize the instance variables of the class. Parameterized constructor is the constructor that takes parameters. It is used to pass the values to the constructor at the time of object creation.

Syntax:

```
class ClassName {  
  // Instance Variables  
  int? number;  
  String? name;  
  // Parameterized Constructor
```



```
    ClassName(this.number, this.name);  
}
```

Eg:

```
class Student {  
    String? name;  
    int? age;  
    int? rollNumber;  
    // Constructor  
    Student(this.name, this.age, this.rollNumber);  
}
```

```
void main(){  
    // Here student is object of class Student.  
    Student student = Student("John", 20, 1);  
    print("Name: ${student.name}");  
    print("Age: ${student.age}");  
    print("Roll Number: ${student.rollNumber}");  
}
```

output:

Name: John

Age: 20

Roll Number: 1

c. Named Constructor In Dart

In most programming languages like java, c++, c#, etc., we can create multiple constructors with the same name. But in Dart, this is not possible. Well, there is a way. We can create multiple constructors with the same name using named constructors. Named constructors improves code readability. It is useful when you want to create multiple constructors with the same name.

Eg:

```
class Student {  
    String? name;  
    int? age;  
    int? rollNumber;  
  
    // Default Constructor  
    Student() {  
        print("This is a default constructor");  
    }  
  
    // Named Constructor  
    Student.namedConstructor(String name, int age, int rollNumber) {  
        this.name = name;  
    }  
}
```

```

    this.age = age;
    this.rollNumber = rollNumber;
  }
}

void main() {
  // Here student is object of class Student.
  Student student = Student.namedConstructor("John", 20, 1);
  print("Name: ${student.name}");
  print("Age: ${student.age}");
  print("Roll Number: ${student.rollNumber}");
}

```

output:

This is a default constructor

Name: John

Age: 20

Roll Number: 1

d. Constant Constructor In Dart

Constant constructor is a constructor that creates a constant object. A constant object is an object whose value cannot be changed. A constant constructor is declared using the keyword `const`. A Constant Constructor is used to create an object whose value cannot be changed. It improves the performance of the program.

There are some Rules For Declaring Constant Constructor In Dart;

All properties of the class must be final.

It does not have any body.

Only a class containing a `const` constructor is initialized using the `const` keyword.

4. Encapsulation

In Dart, Encapsulation means hiding data within a library, preventing it from outside factors. It helps you control your program and prevent it from becoming too complicated. Dart doesn't support keywords like `public`, `private`, and `protected`. Dart uses `_` (underscore) to make a property or method private. The encapsulation happens at library level, not at class level.

You might wonder, what is meant by a library in Dart?

By default, every .dart file is a library. A library is a collection of functions and classes. A library can be imported into another library using the import keyword.

How To Achieve Encapsulation In Dart?

Encapsulation can be achieved by:

- Declaring the class properties as private by using underscore(_).
- Providing public getter and setter methods to access and update the value of private property.

By using encapsulation in Dart, you can hide data and control access to it through getters and setters.

Getter and Setter Methods

Getter and setter methods are used to access and update the value of private property. This allows you to achieve controlled access to values while keeping the internal representation of the data secure.

Getter method: used to access the value of private property.

Setter method: used to update the value of private property.

Eg:

```
class Employee {  
  // Private properties  
  int? _id;  
  String? _name;  
  
  // Getter method to access private property _id  
  int getId() {  
    return _id!;  
  }  
  // Getter method to access private property _name  
  String getName() {  
    return _name!;  
  }  
  // Setter method to update private property _id  
  void setId(int id) {  
    this._id = id;  
  }  
  // Setter method to update private property _name  
  void setName(String name) {
```

```

        this._name = name;
    }

}

void main() {
    // Create an object of Employee class
    Employee emp = new Employee();
    // setting values to the object using setter
    emp.setId(1);
    emp.setName("John");

    // Retrieve the values of the object using getter
    print("Id: ${emp.getId()}");
    print("Name: ${emp.getName()}");
}
output:
Id: 1
Name: John

```

5. Inheritance In Dart

Inheritance is a sharing of behavior between two classes. It allows you to define a class that extends the functionality of another class. The extend keyword is used for inheriting from parent class. : Whenever you use inheritance, it always creates an is-a relation between the parent and child class like Student is a Person, Truck is a Vehicle, Cow is a Animal etc.

Syntax:

```

class ParentClass {
    // Parent class code
}

class ChildClass extends ParentClass {
    // Child class code
}

```

Here, ParentClass is the superclass and ChildClass is the subclass. The ChildClass inherits the properties and methods of the ParentClass.

You might be curious about what a parent class and a child class are. Would you like to know more about these concepts?

Parent Class: The class whose properties and methods are inherited by another class is called parent class. It is also known as base class or super class.

Child Class: The class that inherits the properties and methods of another class is called child class. It is also known as derived class or subclass.

Eg:

```
class Person {
  // Properties
  String? name;
  int? age;

  // Method
  void display() {
    print("Name: $name");
    print("Age: $age");
  }
}

// Here In student class, we are extending the
// properties and methods of the Person class
class Student extends Person {
  // Fields
  String? schoolName;
  String? schoolAddress;

  // Method
  void displaySchoolInfo() {
    print("School Name: $schoolName");
    print("School Address: $schoolAddress");
  }
}

void main() {
  // Creating an object of the Student class
  var student = Student();
  student.name = "John";
  student.age = 20;
  student.schoolName = "ABC School";
  student.schoolAddress = "New York";
  student.display();
  student.displaySchoolInfo();
}

output:
Name: John
Age: 20
School Name: ABC School
School Address: New York
```

Advantages Of Inheritance In Dart

It promotes reusability of the code and reduces redundant code.
It helps to design a program in a better way.
It makes code simpler, cleaner and saves time and money on maintenance.
It facilitates the creation of class libraries.
It can be used to enforce a standard interface to all children classes.

Types Of Inheritance In Dart

Single Inheritance
Multilevel Inheritance
Hierarchical Inheritance
Multiple Inheritance

1. Single Inheritance In Dart

A class (child) inherits from only one superclass (parent). In this type of inheritance, a class can inherit from only one class. In Dart, we can only extend one class at a time.

Eg:

```
class Animal {  
  void sound() {  
    print('Animal sound');  
  }  
}
```

```
class Dog extends Animal {  
  void bark() {  
    print('Bark');  
  }  
}
```

2. Multilevel Inheritance In Dart

A class inherits from another class, which in turn inherits from another class, forming a chain. In this type of inheritance, a class can inherit from another class and that class can also inherit from another class. In Dart, we can extend a class from another class which is already extended from another class.

Eg:

```
class Animal {  
  void sound() {  
    print('Animal sound');  
  }  
}
```

```

    }
}

class Dog extends Animal {
  void bark() {
    print('Bark');
  }
}

class Puppy extends Dog {
  void whimper() {
    print('Whisper');
  }
}

```

3. Multiple Inheritance In Dart

Dart doesn't support multiple inheritance directly, but you can achieve similar functionality using mixins. A mixin is a way to reuse a class's code in multiple class hierarchies.

Eg:

```

mixin CanFly {
  void fly() {
    print('Flying');
  }
}

class Bird with CanFly {
  void chirp() {
    print('Chirp');
  }
}

```

4. Hierarchical Inheritance In Dart

Multiple child classes inherit from a single parent class. In this type of inheritance, a class can inherit from another class and that class can also inherit from another class. In Dart, we can extend a class from another class which is already extended from another class.

Eg:

```

class Animal {
  void sound() {
    print('Animal sound');
  }
}

```

```

    }
}

class Dog extends Animal {
  void bark() {
    print('Bark');
  }
}

class Cat extends Animal {
  void meow() {
    print('Meow');
  }
}

```

Interface Inheritance

Dart allows a class to implement multiple interfaces. An interface in Dart is a class that defines methods without implementing them.

Eg:

```

abstract class Flyer {
  void fly();
}

class Bird implements Flyer {
  @override
  void fly() {
    print('Bird is flying');
  }
}

```

Inheritance Of Constructor In Dart?

Inheritance of constructor in Dart is a process of inheriting the constructor of the parent class to the child class. It is a way of reusing the code of the parent class.

Eg:

```

class Laptop {
  // Constructor
  Laptop() {
    print("Laptop constructor");
  }
}

class MacBook extends Laptop {

```



```
// Constructor
MacBook() {
  print("MacBook constructor");
}
}
```

```
void main() {
  var macbook = MacBook();
}
```

output:

Laptop constructor

MacBook constructor

Super Keyword

Super keyword is important in dart, Super is used to refer to the parent class. It is used to call the parent class's properties and methods.

Eg:

```
class Laptop {
  // Method
  void show() {
    print("Laptop show method");
  }
}
class MacBook extends Laptop {
  void show() {
    super.show(); // Calling the show method of the parent class
    print("MacBook show method");
  }
}
void main() {
  // Creating an object of the MacBook class
  MacBook macbook = MacBook();
  macbook.show();
}
```

output:

Laptop show method

MacBook show method

Super Using in Constructor In Dart

Eg:

```
class Employee {
  // Constructor
  Employee(String name, double salary) {
```

```

    print("Employee constructor");
    print("Name: $name");
    print("Salary: $salary");
  }
}
class Manager extends Employee {
  // Constructor
  Manager(String name, double salary) : super(name, salary) {
    print("Manager constructor");
  }
}
void main() {
  Manager manager = Manager("John", 25000.0);
}
output:
Employee constructor
Name: John
Salary: 25000.0
Manager constructor

```

- **Super Using in Named Constructor In Dart**

Eg:

```

class Employee {
  // Named constructor
  Employee.manager() {
    print("Employee named constructor");
  }
}

class Manager extends Employee {
  // Named constructor
  Manager.manager() : super.manager() {
    print("Manager named constructor");
  }
}

void main() {
  Manager manager = Manager.manager();
}
output:
Employee named constructor
Manager named constructor

```

- **Super Using in Multilevel Inheritance In Dart**

Eg:

```

class Laptop {
    // Method
    void display() {
        print("Laptop display");
    }
}

class MacBook extends Laptop {
    // Method
    void display() {
        print("MacBook display");
        super.display();
    }
}

class MacBookPro extends MacBook {
    // Method
    void display() {
        print("MacBookPro display");
        super.display();
    }
}

void main() {
    var macbookpro = MacBookPro();
    macbookpro.display();
}

```

output:
MacBookPro display
MacBook display
Laptop display

5. Polymorphism

Polymorphism is the ability of an object to take on many forms. In the real world, polymorphism is updating or modifying the feature, function, or implementation that already exists in the parent class.

Polymorphism By Method Overriding

Method overriding is a technique in which you can create a method in the child class that has the same name as the method in the parent class. The method in the child class overrides the method in the parent class.

Syntax:

```

class ParentClass{

```

```

void functionName(){
}
}
class ChildClass extends ParentClass{
@override
void functionName(){
}
}

```

Static In Dart

If you want to define a variable or method that is shared by all instances of a class, you can use the static keyword. Static members are accessed using the class name. It is used for memory management.

Dart Static Variable

A static variable is a variable that is shared by all instances of a class. It is declared using the static keyword. It is initialized only once when the class is loaded. It is used to store the class-level data.

To declare a static variable in Dart, you must use the static keyword before the variable name.

Syntax:

```

class ClassName {
    static dataType variableName;
}

```

How To Access A Static Variable In Dart

You need to use the `ClassName.variableName` to access a static variable in Dart.

Eg:

```

class ClassName {
    static dataType variableName = value;
    // Accessing the static variable inside same class
    void display() {
        print(variableName);
    }
}

```

```

void main() {

```

```
// Accessing static variable outside the class
dataType value =ClassName.variableName;
}
```

Dart Static Method

A static method is shared by all instances of a class. It is declared using the static keyword. You can access a static method without creating an object of the class.

Syntax:

```
class ClassName{
  static returnType methodName(){
    //statements
  }
}
```

Eg:

```
class SimpleInterest {
  static double calculateInterest(double principal, double rate, double time) {
    return (principal * rate * time) / 100;
  }
}
```

```
void main() {
  print(
    "The simple interest is ${SimpleInterest.calculateInterest(1000, 2, 2)}");
}
```

output: The simple interest is 40.0

Enums In Dart

An enum is a special type that represents a fixed number of constant values. An enum is declared using the keyword enum followed by the enum's name.

Syntax:

```
enum enumName {
  constantName1,
  constantName2,
  constantName3,
  ...
  constantNameN
}
```

I'll show you an example of a weekday selection using Enum with a switch condition,

```
enum days {  
  Sunday,  
  Monday,  
  Tuesday,  
  Wednesday,  
  Thursday,  
  Friday,  
  Saturday  
}  
  
void main() {  
  var today = days.Friday;  
  switch (today) {  
    case days.Sunday:  
      print("Today is Sunday.");  
      break;  
    case days.Monday:  
      print("Today is Monday.");  
      break;  
    case days.Tuesday:  
      print("Today is Tuesday.");  
      break;  
    case days.Wednesday:  
      print("Today is Wednesday.");  
      break;  
    case days.Thursday:  
      print("Today is Thursday.");  
      break;  
    case days.Friday:  
      print("Today is Friday.");  
      break;  
    case days.Saturday:  
      print("Today is Saturday.");  
      break;  
  }  
}  
output:  
Today is Friday.
```

Advantages Of Enum In Dart

It is used to define a set of named constants.

Makes your code more readable and maintainable.

It makes the code more reusable and makes it easier for developers.

Characteristics Of Enum

It must contain at least one constant value.
Enums are declared outside the class.
Used to store a large number of constant values.

Abstract Class

Abstract classes are classes that cannot be initialized. It is used to define the behavior of a class that can be inherited by other classes. An abstract class is declared using the keyword `abstract`.

Syntax:

```
abstract class ClassName {  
    //Body of abstract class  
  
    method1();  
    method2();  
}
```

Abstract Method

An abstract method is a method that is declared without an implementation. It is declared with a semicolon (;) instead of a method body.

Syntax

```
abstract class ClassName {  
    //Body of abstract class  
    method1();  
    method2();  
}
```

Why We Need Abstract Class is the Subclasses of an abstract class must implement all the abstract methods of the abstract class. It is used to achieve abstraction in the Dart programming language.

I'll Show you an example with abstract class,

```
abstract class Vehicle {  
    // Abstract method  
    void start();  
    // Abstract method  
    void stop();  
}
```

```
class Car extends Vehicle {
```

```

// Implementation of start()
@override
void start() {
  print('Car started');
}
// Implementation of stop()
@override
void stop() {
  print('Car stopped');
}
}
class Bike extends Vehicle {
  // Implementation of start()
  @override
  void start() {
    print('Bike started');
  }
  // Implementation of stop()
  @override
  void stop() {
    print('Bike stopped');
  }
}
void main() {
  Car car = Car();
  car.start();
  car.stop();

  Bike bike = Bike();
  bike.start();
  bike.stop();
}

```

output:

```

Car started
Car stopped
Bike started
Bike stopped

```

The abstract class is used to define the behavior of a class that can be inherited by other classes. You can define an abstract method inside an abstract class.

Interface In Dart

An interface defines a syntax that a class must follow. It is a contract that defines the capabilities of a class. It is used to achieve abstraction in the Dart programming language.

Syntax:


```
class InterfaceName {  
    // code  
}
```

```
class ClassName implements InterfaceName {  
    // code  
}
```

When you implement an interface, you must implement all the properties and methods defined in the interface. Keyword implements is used to implement an interface.

Declaring Interface In Dart

In dart there is no keyword interface but you can use class or abstract class to declare an interface. All classes implicitly define an interface. Mostly abstract class is used to declare an interface.

Syntax:

```
abstract class Person {  
    canWalk();  
    canRun();  
}
```

You must use the implements keyword to implement an interface. The class that implements an interface must implement all the methods and properties of the interface.

Eg:

```
class Student implements Person {  
    // implementation of canWalk()  
    @override  
    canWalk() {  
        print('Student can walk');  
    }  
}
```

```
// implementation of canRun()  
    @override  
    canRun() {  
        print('Student can run');  
    }  
}
```

Multiple Inheritance In Dart

Multiple inheritance means a class can inherit from more than one class. In dart, you can't inherit from more than one class. But you can implement multiple interfaces in a class.

Syntax:

```
class ClassName implements Interface1, Interface2, Interface3 {  
  // code  
}
```

Eg:

```
// abstract class as interface  
abstract class Area {  
  void area();  
}  
// abstract class as interface  
abstract class Perimeter {  
  void perimeter();  
}  
// implements multiple interfaces  
class Rectangle implements Area, Perimeter {  
  // properties  
  int length, breadth;  
  
  // constructor  
  Rectangle(this.length, this.breadth);  
  
  // implementation of area()  
  @override  
  void area() {  
    print('The area of the rectangle is ${length * breadth}');  
  }  
  // implementation of perimeter()  
  @override  
  void perimeter() {  
    print('The perimeter of the rectangle is ${2 * (length + breadth)}');  
  }  
}  
  
void main() {  
  Rectangle rectangle = Rectangle(10, 20);  
  rectangle.area();  
  rectangle.perimeter();  
}  
output:  
The area of the rectangle is 200  
The perimeter of the rectangle is 60
```

Mixin In Dart

Mixins are a way of reusing the code in multiple classes. Mixins are declared using the keyword `mixin` followed by the mixin name. Three keywords are used while working with mixins: `mixin`, `with`, and `on`. It is possible to use multiple mixins in a class.

There are some rules for Mixin

Mixin can't be instantiated. You can't create object of mixin.

Use the mixin to share the code between multiple classes.

Mixin has no constructor and cannot be extended.

It is possible to use multiple mixins in a class.

Syntax:

```
mixin Mixin1{  
    // code  
}
```

```
mixin Mixin2{  
    // code  
}
```

```
class ClassName with Mixin1, Mixin2{  
    // code  
}
```

Eg:

```
mixin ElectricVariant {  
    void electricVariant() {  
        print('This is an electric variant');  
    }  
}
```

```
mixin PetrolVariant {  
    void petrolVariant() {  
        print('This is a petrol variant');  
    }  
}
```

// with is used to apply the mixin to the class

```
class Car with ElectricVariant, PetrolVariant {  
    // here we have access of electricVariant() and petrolVariant() methods  
}
```

```
void main() {  
    var car = Car();  
}
```

```
car.electricVariant();
car.petrolVariant();
}
```

output:

This is an electric variant

This is a petrol variant

In dart, we can access our constructor in easily in another way which is known as factory constructor. All of the constructors that you have learned until now are generative constructors. Dart also provides a special type of constructor called a factory constructor.

A factory constructor gives more flexibility to create an object. Generative constructors only create an instance of the class. But, the factory constructor can return an instance of the class or even subclass. It is also used to return the cached instance of the class.

Syntax:

```
class ClassName {
  factory ClassName() {
    // TODO: return ClassName instance
  }

  factory ClassName.namedConstructor() {
    // TODO: return ClassName instance
  }
}
```

Eg:

Without Factory Constructor

```
class Area {
  final int length;
  final int breadth;
  final int area;
```

Eg:

With Factory Constructor

```
class Area {
  final int length;
  final int breadth;
  final int area;
```

```
// private constructor
```

```
// Initializer list
const Area(this.length, this.breadth) : area =
length * breadth;
}
```

```
void main() {
  Area area = Area(10, 20);
  print("Area is: ${area.area}");

  // notice that here is a negative value
  Area area2 = Area(-10, 20);
  print("Area is: ${area2.area}");
}
```

output:

```
Area is: 200
Area is: -200
```

```
const Area._internal(this.length, this.breadth) :
area = length * breadth;
```

```
// Factory constructor
factory Area(int length, int breadth) {
  if (length < 0 || breadth < 0) {
    throw Exception("Length and breadth must be
positive");
  }
  // redirect to private constructor
  return Area._internal(length, breadth);
}
}
```

```
void main() {
  // This works
  Area area = Area(10, 20);
  print("Area is: ${area.area}");
```

```
// notice that here is negative value
Area area2 = Area(-10, 20);
print("Area is: ${area2.area}");
}
```

output:

```
Area is: 200
Unhandled exception:
Exception: Length and breadth must be positive
```

Here also some rules to for Factory Constructors

- Factory constructors must return an instance of the class or sub-class.
- You can't use this keyword inside a factory constructor.
- It can be named or unnamed and called like a normal constructor.
- It can't access instance members of the class.

Null Safety

Null safety is a feature in the Dart programming language that helps developers to avoid null errors. This feature is called Sound Null Safety in dart. This allows developers to catch null errors at edit time. Null safety avoids null errors, runtime bugs, vulnerabilities, and system crashes which are difficult to find and fix.

Advantage Of Null Safety

- Write safe code.
- Reduce the chances of application crashes.
- Easy to find and fix bugs in code.

How To Declare Null Value

Syntax:

String? name;

Eg:

```
void main(){  
  // Declaring a nullable variable by using ?  
  String? name;  
  // Assigning John to name  
  name = "John";  
  // Assigning null to name  
  name = null;  
}
```

Late Keyword In Dart

Late keyword is used to declare a variable or field that will be initialized at a later time. It is used to declare a non-nullable variable that is not initialized at the time of declaration.

Eg:

late String name;

```
void main() {  
  // assigning value to late variable  
  name = "John";  
  print(name);  
}  
output: John
```

The late keyword is a contract between you and Dart. You are telling Dart that you will assign a value to the variable before you use it. If you don't assign a value to the variable before you use it, Dart will throw an error.

There have some conditions to use late keyword in dart

- Don't assign that variable a value yet.
- You will assign value later.
- You will make sure the variable has a value before you use it.

Usecase of Late Keyword In Dart

Dart late keyword has two use cases:

- Declaring a non-nullable variable or field that is not initialized at the point of declaration.
- Lazy initialization of a variable or field.

Using late means dart doesn't initialize value right away, it only initializes when you access it for the first time. This is also called lazy loading.

Eg:

```
// function
String provideCountry() {
  print("Function is called");
  return "USA";
}

void main() {
  print("Starting");
  // late variable
  late String value = provideCountry();
  print("End");
  print(value);
}
```

output:

Starting

End

Function is called

USA

1. Asynchronous Programming In Dart

Asynchronous Programming is a way of writing code that allows a program to do multiple tasks at the same time. Time consuming operations like fetching data from the internet, writing to a database, reading from a file, and downloading a file can be performed without blocking the main thread of execution.

2. Synchronous Programming in Dart

In Synchronous programming, the program is executed line by line, one at a time. Synchronous operation means a task that needs to be solved before proceeding to the next one.

Difference between Asynchronous and Synchronous Programming

Eg:

Asynchronous Programming

```
void main() {  
  print("First Operation");  
  
  Future.delayed(Duration(seconds:3),()=>print('Second Big Operation'));  
  print("Third Operation");  
  print("Last Operation");  
}
```

output:

First Operation
Third Operation
Last Operation
Second Big Operation

Eg:

Synchronous Programming

```
void main() {  
  print("First Operation");  
  print("Second Big Operation");  
  print("Third Operation");  
  print("Last Operation");  
}
```

output:

First Operation
Second Big Operation
Third Operation
Last Operation

Needs of Asynchronous

- To Fetch Data From Internet,
- To Write Something to Database,
- To execute a long-time consuming task,
- To Read Data From File, and
- To Download File etc.

Such asynchronous operations usually take a long time to complete, so it usually provides results in the form of a Future. If the result has multiple parts, then it is provided as a Stream. You will learn about Future and Stream in the next section.

Future In Dart

In dart, the Future represents a value or error that is not yet available. It is used to represent a potential value, or error, that will be available at some time in the future.

Create Future In Dart

You can create a future in dart by using the Future class. Here the function will return Future<String> after 5 seconds.

Eg:

```
// function that returns a future
Future<String> getUsername() {
  return Future.value('Mark');
}
```

How To Use Future In Dart

Eg:

```
Future<String> getUsername() async {
  return Future.delayed(Duration(seconds: 2), () => 'Mark');
}
```

```
// main function
void main() {
  print("Start");
  getUsername().then((value) => print(value));
  print("End");
}
```

output:

Start

End

Mark

Future represents the result of an asynchronous operation and can have 2 states.

- Uncompleted
- Completed
- Uncompleted

When you call an asynchronous function, it returns to an uncompleted future. It means the future is waiting for the function asynchronous operation to finish or to throw an error.

Completed

It can be completed with value or completed with error. `Future<int>` produces an int value, and `Future<String>` produces a String value. If the future doesn't produce any value, then the type of future is `Future<void>`.

Async And Await In Dart

`Async/await` is a feature in Dart that allows us to write asynchronous code that looks and behaves like synchronous code, making it easier to read. When a function is marked `async`, it signifies that it will carry out some work that could take some time and will return a `Future` object that wraps the result of that work.

Await

The `await` keyword, on the other hand, allows you to delay the execution of an `async` function until the awaited `Future` has finished. This enables us to create code that appears to be synchronous but is actually asynchronous.

Async

The `async` and `await` keywords both provide a declarative way to define an asynchronous function and use their results. You can use the `async` keyword before a function body to make it asynchronous. You can use the `await` keyword to get the completed result of an asynchronous expression.

Condition

To define an Asynchronous function, add `async` before the function body. The `await` keyword work only in the `async` function.

Eg:

Synchronous Function

```
void main() {  
  print("Start");  
  getData();  
  print("End");  
}
```

Eg:

Asynchronous Function

```
void main() {  
  print("Start");  
  getData();  
  print("End");  
}
```

```
void getData() {
  String data = middleFunction();
  print(data);
}
```

```
Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=>
    "Hello");
}
output:
Start
End
Instance of '_Future<String>'
```

```
void getData() async{
  String data = await middleFunction();
  print(data);
}
```

```
Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=>
    "Hello");
}
output:
Start
End
Hello
```

Handling Errors

You can handle errors in the dart async function by using try-catch. You can write try-catch code the same way you write synchronous code.

Eg:

```
main() {
  print("Start");
  getData();
  print("End");
}
```

```
void getData() async{
  try{
    String data = await middleFunction();
    print(data);
  }catch(err){
    print("Some error $err");
  }
}

Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=> "Hello");
}
output:
Start
End
Hello
```

Streams in Dart

A stream is a sequence of asynchronous events representing multiple values that will arrive in the future. Stream class deals with sequences of events instead of single events. Stream has one or more listeners, and all listeners will receive the same value.

You can create a stream in dart by using Stream class. Here the function will return Stream<String> after 5 seconds.

Eg:

```
// function that returns a stream
Stream<String> getUsername() async* {
  await Future.delayed(Duration(seconds: 1));
  yield 'Mark';
  await Future.delayed(Duration(seconds: 1));
  yield 'John';
  await Future.delayed(Duration(seconds: 1));
  yield 'Smith';
}
```

Here yield returns the value from the stream. To use yield you have to use async*. You can also create a stream by using Stream.fromIterable() method. Here the function will return Stream<String> immediately.

Syntax:

```
// function that returns a stream
Stream<String> getUsername() {
  return Stream.fromIterable(['Mark', 'John', 'Smith']);
}
```

How To Use Stream In Dart

You can use stream in dart by using "await" for loop.

Eg:

```
// function that returns a stream
Stream<String> getUsername() async* {
  await Future.delayed(Duration(seconds: 1));
  yield 'Mark';
  await Future.delayed(Duration(seconds: 1));
  yield 'John';
  await Future.delayed(Duration(seconds: 1));
  yield 'Smith';
}
```

```

}

// main function
void main() async {
  // you can use await for loop to get the value from stream
  await for (String name in getUsername()) {
    print(name);
  }
}

```

output:
Mark
John
Smith

Difference b/w Future and Stream.

Future represents the value or error that is supposed to be available in the Future.
Stream is a way by which we receive a sequence of events.

A Future can provide only a single result over time.	Stream can provide zero or more values.
You can use FutureBuilder to view and interact with data.	You can use StreamBuilder to view and interact with data.
It can't listen to a variable change.	But Stream can listen to a variable change.
Syntax: Future <data_type> class_name	Syntax: Stream <data_type> class_name

Types Of Stream

There are two types of streams:

- Single Subscription streams
- Broadcast streams

Single Subscription Stream

By default, Streams are set up for a single subscription. They hold onto the values until someone subscribes and can only be listened to once. You will get an exception if you try to listen more than once. Any event's value should not be missed and must be in the

correct order. Inside the stream controller, there is only one stream, and only one subscriber can use that stream.

Broadcast Stream

This is the stream that is set up for multiple subscriptions. They hold onto the values until subscribers can only listen many times. You can use the broadcast stream if you want more objects to listen to the stream. It can be used for mouse events in a browser. Inside the stream controller, many streams can be used by many subscribers.

Syntax:

```
StreamController<data_type> controller = StreamController<data_type>.broadcast();
```

We can Create Stream like,

```
StreamController<data_type> controller = StreamController<data_type>();  
Stream stream = controller.stream;
```

Now that we've completed our Dart documentation section, it's time to start learning Flutter.

Here is how we can use Future and async in dart

async It gives a Future. async keyword does some work that might take a long time. It gives the result wrapped in future.	async* It gives a Stream. async* returns a bunch of future values on at a time. It gives the result wrapped in the stream.
---	--